

Escenario, Diseño de Arquitectura y Contrato de Integración

Introducción

El presente documento tiene como objetivo describir el escenario, la arquitectura y el contrato de integración del Sistema de Prevención de Fuga de Datos en el Uso de Inteligencia Artificial mediante Intervención Contextual en Tiempo Real, orientado a Pequeñas y Medianas Empresas (PYMES) argentinas. Se abordan, en primer lugar, la situación de vulnerabilidad de las PYMES regionales frente a la ciberseguridad; en segundo lugar, el fenómeno de Shadow AI y los riesgos de fuga de datos asociados al uso no supervisado de herramientas de Inteligencia Artificial Generativa; en tercer lugar, el marco normativo argentino vigente en materia de protección de datos; en cuarto lugar, el fundamento de la intervención contextual como estrategia de mitigación frente al bloqueo genérico; en quinto lugar, las bases tecnológicas de las extensiones de navegador como capa de intercepción; y finalmente, la arquitectura general del sistema, el contrato de comunicación entre sus componentes y el diseño de la interfaz de intervención y del dashboard de auditoría, resultado del trabajo de diseño realizado antes de avanzar hacia la etapa de implementación y validación empírica.

Etapas 1

Investigación de la API de extensiones de arquitecturas y frameworks de dashboard.

API de extensiones de navegador - Manifest V3

Desde 2023 Google discontinuó Manifest V2 y, a partir de 2026, el Chrome Web Store ya no acepta nuevas publicaciones bajo ese formato, por lo que Manifest V3 (MV3) es la única especificación vigente para el desarrollo de extensiones de Chrome. MV3 reorganiza la arquitectura de una extensión en torno a cuatro piezas declaradas explícitamente en el archivo manifest.json: el service worker, los content scripts, la interfaz propia de la extensión (popup u options page) y los permisos solicitados.

El cambio estructural más relevante frente a la especificación anterior es el reemplazo de las páginas de fondo persistentes por un service worker de vida limitada: este proceso se activa ante un evento (una interacción del usuario, una petición de red, una alarma) y se termina automáticamente tras un período de inactividad, lo que impide almacenar estado en variables globales y obliga a persistir cualquier dato necesario mediante la API chrome.storage. Asimismo, MV3 reemplazó la API bloqueante webRequest por declarativeNetRequest, un modelo de reglas declarativas para la gestión de peticiones de

red, y restringió la ejecución de código remoto, lo que redujo la superficie de ataque de las extensiones.

Para este proyecto, la pieza central es el content script: un script inyectado en el contexto de la página visitada, con acceso al DOM pero no al contexto de JavaScript de la propia página (por lo que la comunicación con el código de la página, de ser necesaria, requiere mecanismos como `window.postMessage`). El content script es el componente responsable de detectar los formularios de entrada de ChatGPT, Gemini y Claude, interceptar el evento de envío, y comunicar el texto capturado hacia el service worker mediante el sistema de paso de mensajes de la extensión (`chrome.runtime.sendMessage`). Dado que los content scripts no pueden realizar peticiones de red directamente hacia dominios externos bajo MV3, es el service worker quien concentra la comunicación HTTPS con el backend de análisis.

Frameworks para el dashboard de administración

Para la selección del framework del panel de administración se relevaron las tres alternativas dominantes del ecosistema frontend: React, Angular y Vue. Los tres son opciones maduras y con rendimiento adecuado para el caso de uso, pero difieren en el modelo de desarrollo que proponen: React se orienta a la libertad arquitectónica y a un ecosistema amplio de librerías complementarias; Angular ofrece una estructura más opinada y convenciones oficiales, lo que lo vuelve una opción frecuente para dashboards empresariales grandes con equipos numerosos; y Vue prioriza una curva de aprendizaje más suave y una adopción incremental.

Se optó por React para este proyecto por tres motivos concretos: (a) es actualmente el framework con mayor adopción y comunidad del ecosistema frontend, lo que facilita la resolución de problemas durante el desarrollo; (b) su modelo de componentes y de estado unidireccional se ajusta bien a una vista que debe re-renderizarse a partir de eventos que llegan de forma asíncrona desde el backend; y (c) cuenta con librerías de visualización de datos (para tablas filtrables y gráficos de eventos por tipo de dato, usuario o IA de destino) que se integran de forma directa con este modelo de componentes, sin requerir una curva de aprendizaje adicional significativa para el resto del equipo.

Redacción del escenario de simulación y especificación de los casos de uso del sistema

Escenario

Contexto: una PyME ficticia del sector servicios, sin equipo de IT propio, en la que un conjunto reducido de empleados con distintos perfiles utiliza asistentes de IA en su trabajo diario. Se definen tres perfiles de empleado para las pruebas:

- Perfil administrativo: redacta correos y presupuestos, con riesgo de exponer datos de clientes (CUIT, CBU, datos de contacto).
- Perfil de desarrollo: utiliza IA para depurar código, con riesgo de exponer credenciales, claves de API o código propietario.
- Perfil de gestión: utiliza IA para resumir informes internos, con riesgo de exponer información financiera o estratégica de la empresa.

Para cada perfil se diseñarán casos de prueba combinando: (a) texto sin datos sensibles, (b) texto con un único tipo de dato sensible, (c) texto con código propietario, y (d) combinaciones mixtas de varios tipos de datos sensibles en un mismo envío. Se comparará, para un subconjunto de los casos, el comportamiento del usuario ante una intervención contextual explicativa frente a un mensaje de bloqueo genérico sin justificación, registrando la tasa de aceptación de la alerta (edición o cancelación del envío) frente a la tasa de continuación bajo riesgo. Estos resultados alimentarán la matriz de resultados prevista como entregable de la Etapa 4.

Especificación de Casos de Uso

Nº	Requerimiento	Componente
RF-01	Detectar la carga de las páginas de ChatGPT, Gemini y Claude e identificar el formulario de entrada de texto.	Extensión
RF-02	Interceptar el evento de envío del formulario antes de que el texto sea transmitido al servicio de IA.	Extensión
RF-03	Enviar el texto capturado al backend de análisis a través de una petición HTTPS y esperar la respuesta de riesgo.	Extensión
RF-04	Permitir el envío original del texto cuando el backend determina riesgo bajo o nulo.	Extensión
RF-05	Bloquear el envío y desplegar un modal de intervención contextual cuando el backend determina riesgo medio o alto.	Extensión
RF-06	Mostrar en el modal una explicación en lenguaje simple del tipo de dato detectado y el motivo del riesgo.	Extensión
RF-07	Permitir al usuario decidir la acción a seguir (editar el texto, cancelar el envío o continuar bajo su responsabilidad).	Extensión
RF-08	Registrar cada evento evaluado (usuario, tipo de dato detectado, IA de destino, nivel de riesgo, acción tomada).	Dashboard
RF-09	Listar y filtrar los eventos registrados por usuario, tipo de dato, IA de destino y rango de fechas.	Dashboard

RF-10	Presentar métricas agregadas (cantidad de eventos por nivel de riesgo, por IA de destino, por usuario) mediante visualizaciones.	Dashboard
RF-11	Recibir el texto enviado por la extensión y aplicar un conjunto de reglas por expresiones regulares sobre patrones estructurados (CUIT/CUIL, CBU, DNI, teléfono, correo electrónico, credenciales).	Backend
RF-12	Aplicar procesamiento de lenguaje natural (reconocimiento de entidades nombradas con spaCy) para detectar nombres propios y organizaciones no capturables mediante patrones fijos.	Backend
RF-13	Detectar código fuente propietario combinando sintaxis de código con una lista configurable de términos propios de la empresa.	Backend
RF-14	Clasificar el nivel de riesgo del texto analizado en cuatro categorías (bajo, medio, alto, crítico) y generar el mensaje contextual explicativo asociado a cada nivel.	Backend
RF-15	Persistir cada evento evaluado (incluidos los de riesgo bajo) en una base de datos, junto con el resultado del análisis.	Backend
RF-16	Exponer los endpoints REST (POST /analizar , PATCH /eventos/{id} , GET /eventos) consumidos por la extensión y el dashboard.	Backend

Definición conjunta de la estructura de datos, payloads y flujo de comunicación entre la extensión de navegador y el motor de análisis.

A partir de los casos de uso especificados, ambas integrantes consolidaron el contrato de datos entre la extensión y el backend de análisis, formalizado como modelos de datos tipados en el backend. El contrato define tres intercambios:

1. Solicitud de análisis (POST /analizar**)** — enviada por el service worker de la extensión cada vez que se intercepta un intento de envío:

```
{
  "texto": "texto capturado por el content script antes del envío",
  "usuario_id": "id persistido en chrome.storage",
  "ia_destino": "chatgpt | gemini | claude"
}
```

2. Respuesta del análisis — devuelta por el backend con la decisión de riesgo:

```
{
  "evento_id": "id generado por el backend",
}
```

```
"nivel_riesgo": "bajo | medio | alto | critico",
"tipo_dato_detectado": "ej. CUIT/CUIL, DNI, Teléfono, Nombre de persona (o null)",
"fragmento_detectado": "fragmento del texto que disparó la regla (o null)",
"mensaje_contextual": "explicación en lenguaje simple del riesgo (o null)"
}
```

3. Actualización de la acción tomada (PATCH /eventos/{evento_id}) — enviada por la extensión una vez que el usuario decide qué hacer frente al modal:

```
{
  "accion": "permitido_automatico | editar | cancelar | continuar"
}
```

Este contrato se acordó tomando como criterio de diseño que el `usuario_id` y el `evento_id` fueran identificadores opacos generados y persistidos del lado de cada componente (el primero por la extensión, el segundo por el backend), de modo que ninguno de los dos lados necesitara conocer detalles internos del otro. El flujo de comunicación resultante queda así definido en tres pasos: la extensión envía el texto y recibe la clasificación de riesgo; si corresponde, muestra el modal y captura la decisión del usuario; y notifica esa decisión al backend para dejar registrado el evento completo, consultable luego por el dashboard de administración.

Etapas 2

Diseño de la arquitectura

Estilo arquitectónico

El sistema se organiza siguiendo el patrón de arquitectura de tres capas (three-tier architecture), un estilo consolidado en ingeniería de software que separa una aplicación en capa de presentación, capa de lógica de negocio y capa de datos, cada una desplegable y escalable de forma independiente. En este proyecto esa separación se traduce en tres componentes desacoplados y físicamente distintos: la extensión de navegador (capa de presentación, corre en el dispositivo del usuario), el backend de análisis en FastAPI (capa de lógica, aplica las reglas y el NLP que determinan el riesgo) y el dashboard en React (una segunda capa de presentación, orientada a la auditoría), con el registro de eventos del propio backend actuando como capa de datos compartida entre ambos frontends. Este desacoplamiento es el que permite, en términos de la arquitectura de tres capas, que cada componente pueda modificarse o escalarse sin impactar a los demás (por ejemplo, cambiar el motor de NLP del backend no requiere tocar la extensión).

Comunicación entre componentes: estilo REST

La comunicación entre la extensión, el backend y el dashboard se implementa como una API REST (Representational State Transfer), el estilo arquitectónico definido por Roy Fielding en su tesis doctoral de 2000, que propone tratar cada recurso del sistema (en este caso, un evento de análisis) como una entidad direccionable mediante una URL, manipulada a través de verbos HTTP estándar (POST para crear un evento de análisis, GET para consultarlos, PATCH para actualizar la acción tomada por el usuario). Se optó por REST en lugar de un protocolo con estado persistente (como WebSockets) porque la interacción del sistema es de tipo solicitud-respuesta puntual (el usuario envía un texto, espera una decisión) y no requiere una conexión abierta permanente, lo que simplifica la implementación y es coherente con el enfoque liviano y de bajo costo del proyecto.

Arquitectura interna de cada componente

- *Extensión (Manifest V3)*: dividida en content script (intercepción del DOM y despliegue del modal) y service worker (comunicación HTTPS con el backend), según el modelo estándar de extensiones de Chrome descrito en la Etapa 1.
- *Backend (FastAPI)*: expone el endpoint REST que orquesta el motor de reglas/regex/NLP y persiste los eventos, a cargo de Denise Pujalte.
- *Dashboard (React)*: consume la misma API REST del backend mediante peticiones GET, sin acceso directo a la base de eventos, respetando el principio de separación de capas.

Flujo completo

1. El usuario escribe texto en ChatGPT, Gemini o Claude y presiona "Enviar".
2. El content script de la extensión intercepta el evento de envío (`preventDefault`) antes de que el texto salga hacia el servicio de IA.
3. El service worker envía el texto al backend mediante `POST /analizar`, con el texto, el `usuario_id` y la `ia_destino`.
4. El backend aplica el motor de reglas, expresiones regulares y NLP para calcular el `nivel_riesgo` y el `tipo_dato_detectado`.
5. El backend registra el evento evaluado (incluidos los de riesgo bajo, para mantener un universo completo de eventos).
6. El backend responde con la decisión: `nivel_riesgo`, `tipo_dato_detectado` y `mensaje_contextual`.
7. La extensión evalúa la decisión recibida.
8. Si el riesgo es bajo, permite el envío original del texto sin interrupción visible.
9. Si el riesgo es medio o alto, bloquea el envío y despliega el modal de intervención contextual con la explicación en lenguaje simple.
10. El usuario decide: editar el texto, cancelar el envío, o continuar bajo su responsabilidad; la extensión notifica esa decisión al backend mediante `PATCH /eventos/{id}`.

11. El dashboard consulta `GET /eventos` para mostrar el listado y las métricas agregadas al responsable de gestión.

Diseño de la Interfaz de la Extensión: Modal de Alerta, Mensajes por Tipo de Dato y Opciones de Acción

A partir del contrato de comunicación definido, se diseñó el modal de intervención contextual que la extensión despliega sobre el formulario de la herramienta de IA cuando el backend determina un nivel de riesgo medio, alto o crítico (cumpliendo RF-05 y RF-06). El diseño contempla los siguientes elementos, visibles en la Figura 1:

- **Encabezado con severidad:** el título ("Este mensaje contiene información sensible") se acompaña de una etiqueta de color que indica el nivel de riesgo detectado (por ejemplo, "Riesgo alto"), y se identifica la herramienta de IA de destino (en el ejemplo, Gemini).
- **Tipo de dato detectado:** se muestra de forma explícita la categoría del dato sensible identificado (por ejemplo, "CUIT/CUIL"), evitando un mensaje genérico de bloqueo.
- **Fragmento detectado:** se resalta la porción exacta del texto que disparó la alerta, para que el usuario ubique de inmediato qué debe corregir sin tener que releer todo el mensaje.
- **Explicación en lenguaje simple ("Por qué te lo mostramos"):** fundamenta el riesgo citando la normativa aplicable (Ley 25.326), traduciendo el criterio legal relevado en el marco teórico a un lenguaje comprensible para un usuario no técnico.
- **Opciones de acción (RF-07):** tres botones — "Editar el texto", "Cancelar envío" y "Continuar de todos modos" — permiten al usuario decidir cómo proceder. Para los niveles de riesgo alto y crítico, la opción "Continuar de todos modos" se deshabilita (como se observa en la figura), forzando al usuario a editar o cancelar; para el nivel medio, las tres opciones permanecen disponibles.
- **Nota de refuerzo:** un texto al pie aclara cuál es la opción recomendada y advierte que "Continuar de todos modos" queda registrado en la auditoría, reforzando la trazabilidad de la decisión del usuario sin impedirla en los casos donde sí está habilitada.

Este diseño prioriza la comprensión del riesgo por sobre la restricción pura, coherente con el principio de intervención contextual que fundamenta el proyecto: el usuario recibe la información necesaria para corregir su propio comportamiento, en lugar de encontrarse con un bloqueo sin justificación.

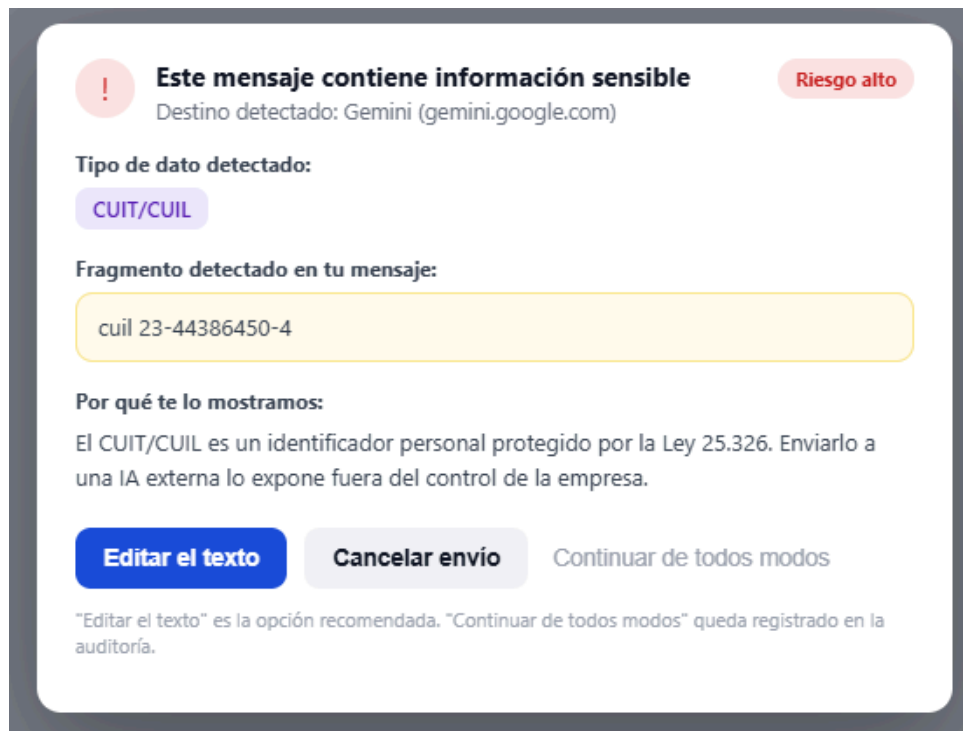


Figura 1.Modal de alerta

Documentación de la arquitectura técnica, diagramas de componentes y especificaciones de cada módulo del sistema.

Módulo	Subsistema	Responsabilidad	Tecnología	Depende de
Content Script	Extensión	Detectar el formulario de IA en la página, interceptar el evento de envío y resaltar el fragmento de texto detectado dentro del modal.	JavaScript (inyectado vía Manifest V3)	Service Worker (mensajería interna)
Service Worker	Extensión	Orquestrar la petición HTTPS hacia el backend, persistir configuración local (usuario_id) y enrutar mensajes entre el content script y la red externa.	JavaScript, chrome.storage, fetch	API REST del backend (IAAnálisis)
Interfaz de Intervención	Extensión	Renderizar el modal contextual con el mensaje explicativo y capturar la acción elegida por el usuario.	HTML/CSS/J S inyectado en el DOM	Service Worker (recibe la decisión)

API REST	Backend	Exponer los endpoints POST /analizar, PATCH /eventos/{id} y GET /eventos; validar y enrutar cada solicitud.	Python, FastAPI	Motor de Detección, Registro de Eventos
Motor de Detección	Backend	Aplicar reglas, expresiones regulares y NLP (spaCy) sobre el texto recibido para calcular nivel_riesgo, tipo_dato_detectado y mensaje_contextual.	Python, spaCy	Ninguno (módulo autocontenido)
Registro de Eventos	Backend	Persistir cada evento evaluado (incluidos los de riesgo bajo) y exponer el historial consultable por el Dashboard.	Base de datos (a definir en Etapa 3)	Ninguno
Cliente API	Dashboard	Consumir GET /eventos, traduciendo los filtros de la interfaz (usuario, tipo de dato, IA, fecha) a parámetros de consulta.	JavaScript, React	API REST del backend (IEventos)
Listado de Eventos	Dashboard	Renderizar la tabla filtrable de eventos registrados.	React (componentes de tabla)	Cliente API
Panel de Métricas	Dashboard	Renderizar gráficos agregados por nivel de riesgo, IA de destino y usuario.	React + librería de visualización	Cliente API

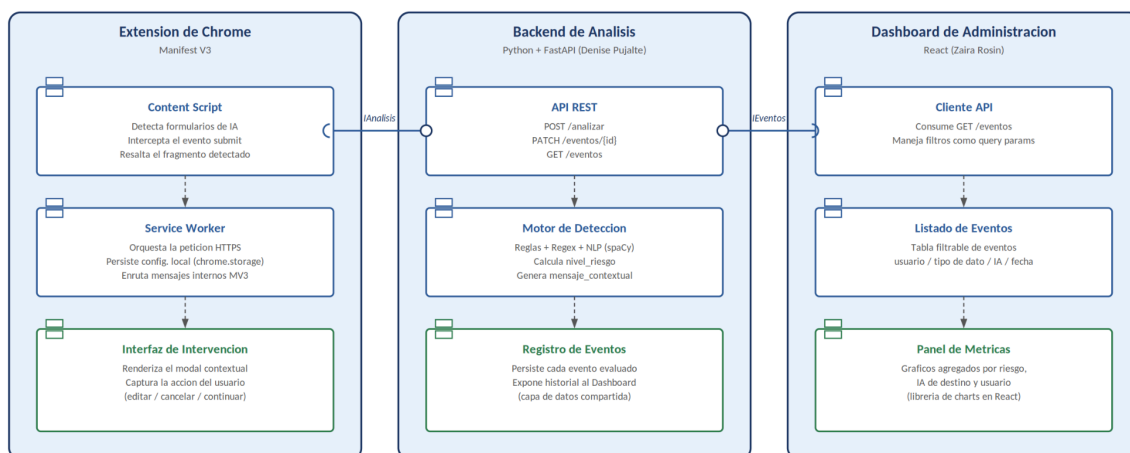


Figura 2. Diagrama de componentes del sistema

Consolidación y validación del contrato de la API y el flujo de decisión (riesgo bajo / medio / alto) entre el frontend y el backend.

Una vez definidos los payloads de comunicación y diseñada la interfaz de intervención, ambas integrantes consolidaron y validaron en conjunto el contrato final antes de pasar a la etapa de implementación. Esta validación se plasmó en el diagrama de componentes de la Figura 4, que representa la arquitectura completa del sistema en notación UML: la Extensión de Chrome (con sus módulos Content Script, Service Worker e Interfaz de Intervención), el Backend de Análisis (con la API REST, el Motor de Detección y el Registro de Eventos) y el Dashboard de Administración (con el Cliente API, el Listado de Eventos y el Panel de Métricas), comunicados mediante dos interfaces formales: **IAnálisis**, que expone el backend hacia la extensión, e **IEventos**, que expone hacia el dashboard.

Sobre esta base, se validó conjuntamente el flujo de decisión que determina el comportamiento de la extensión según el nivel de riesgo devuelto por el backend:

- **Riesgo bajo:** el backend registra el evento igualmente (para mantener un universo completo de eventos), pero responde indicando que no hay dato sensible; la extensión permite el envío original sin ninguna interrupción visible para el usuario.
- **Riesgo medio:** la extensión bloquea el envío y despliega el modal de intervención contextual con las tres opciones de acción disponibles (editar, cancelar o continuar bajo su responsabilidad).
- **Riesgo alto o crítico:** la extensión bloquea el envío y despliega el mismo modal, pero con la opción "Continuar de todos modos" deshabilitada, forzando al usuario a editar el texto o cancelar el envío.

Esta validación conjunta del contrato y del flujo de decisión —realizada antes de que cada integrante avanzara por separado con la implementación de su componente— es lo que permitió que, al momento de integrar la extensión con el backend en la Etapa 3, ambos lados coincidieran sin necesidad de renegociar el formato de los datos ni el criterio de bloqueo por nivel de riesgo.

Etapa 3

La Etapa 3 corresponde a la implementación funcional de los tres componentes diseñados en la Etapa 2: la extensión de navegador, el canal de comunicación hacia el backend, el módulo de scoring de riesgo, y el dashboard de administración. A diferencia de las etapas anteriores, centradas en el relevamiento teórico y el diseño de arquitectura, esta etapa produce artefactos de software ejecutables, verificados de punta a punta contra el contrato de comunicación consolidado en la Sección "Consolidación y validación del contrato de la API".

Desarrollo de la extensión: interceptación de texto en formularios de IAs externas (ChatGPT, Gemini, Claude)

Siguiendo el modelo de content script y service worker descrito en la Etapa 1, se implementó la interceptación sobre los tres sitios objetivo mediante un archivo de configuración declarativo (site-adapters.js) que centraliza, por sitio, el selector del composer de texto y el selector del botón de envío. Esta separación evita hardcodear el DOM de cada sitio dentro del content script y facilita actualizar los selectores cuando alguno de los sitios modifique su interfaz, algo esperable dado que ninguno expone una API estable para este propósito.

```
window.__dlpSiteAdapters = [
  {
    id: "chatgpt",
    hostMatch: (host) => host.includes("chatgpt.com"),
    composerSelector: "#prompt-textarea, form textarea",
    sendButtonSelector: "[data-testid='send-button'], button[aria-label*='Send' i]",
    getText: (el) => (el.value !== undefined ? el.value : el.innerText) || "",
  },
  // ... adapters análogos para gemini.google.com y claude.ai
];
```

Fragmento de site-adapters.js: configuración declarativa por sitio soportado.

El content script escucha, en fase de captura sobre document, tanto el evento click sobre el botón de envío como el evento keydown de la tecla Enter dentro del composer. Al detectar cualquiera de los dos, se invoca preventDefault() y stopImmediatePropagation() antes de que el texto salga hacia el servicio de IA, cumpliendo el requerimiento RF-02. El texto capturado se obtiene mediante la función getText del adapter correspondiente, que abstrae la diferencia entre los compositores basados en textarea (ChatGPT) y los basados en contenteditable (Gemini, Claude).

```
async function manejarIntentoDeEnvio(event) {
  const composer = getComposerEl();
  const texto = (adapter.getText(composer) || "").trim();
  if (!texto) return;

  event.preventDefault();
  event.stopImmediatePropagation();

  const resultado = await analizarTexto(texto);
  if (resultado.nivel_riesgo === "bajo") {
    reenviarDeVerdad(); // RF-04
  } else {
    mostrarModal(resultado); // RF-05 / RF-06 / RF-07
  }
}
```

Fragmento de content-script.js: interceptación del envío y derivación según nivel de riesgo.

Cuando el usuario aprueba el envío desde el modal (acción "continuar") o el backend determina riesgo bajo, la extensión reproduce el envío original disparando un click programático sobre el botón de envío del sitio, evitando así reimplementar la lógica de envío propia de cada plataforma de IA.

Implementación del canal de comunicación entre la extensión y el backend FastAPI

El service worker concentra toda la comunicación HTTPS hacia el backend, tal como establece el modelo de Manifest V3 (los content scripts no pueden realizar peticiones de red directamente hacia dominios externos). Se implementó un único punto de integración, la función callBackend, que envía el POST /analizar con el contrato consolidado en la Etapa 2 (texto, usuario_id, ia_destino) y devuelve la respuesta con nivel_riesgo, tipo_dato_detectado, fragmento_detectado y mensaje_contextual.

```
async function callBackend(texto, usuarioid, iaDestino) {
  const resp = await fetch(`${BACKEND_BASE_URL}/analizar`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ texto, usuario_id: usuarioid, ia_destino: iaDestino }),
  });
  if (!resp.ok) throw new Error("Error del backend: " + resp.status);
  return await resp.json();
}
```

Fragmento de service-worker.js: único punto de integración con el backend.

El identificador de usuario (usuario_id) se genera una única vez por instalación de la extensión y se persiste mediante la API chrome.storage, en lugar de una variable en memoria, dado que el service worker de Manifest V3 tiene una vida limitada y se termina automáticamente tras un período de inactividad. Adicionalmente, se implementó una estrategia de degradación: si la petición al backend falla (por ejemplo, por no estar disponible durante el desarrollo), la extensión no bloquea al usuario, sino que permite el envío original y registra la falla en consola, priorizando no interrumpir el flujo de trabajo del empleado.

Desarrollo del módulo de scoring de riesgo y generación de mensajes contextuales explicativos

El motor de análisis se implementó en Python sobre FastAPI, exponiendo el endpoint POST /analizar descrito en el contrato. La primera capa del motor, a cargo de Denise Pujalte, aplica un conjunto de reglas por expresiones regulares sobre los patrones estructurados relevados en el marco conceptual (CUIT/CUIL, CBU, credenciales o tokens de acceso, correos electrónicos), cada una asociada a un nivel de riesgo y a un mensaje contextual redactado en lenguaje simple, fundamentado en la Ley 25.326.

```

REGLAS: list[Regla] = [
    Regla(
        tipo="CUIT/CUIL",
        patron=re.compile(r"\b\d{2}-?\d{8}-?\d\b"),
        nivel="alto",
        mensaje=(
            "El CUIT/CUIL es un identificador personal protegido por la Ley 25.326. "
            "Enviarlo a una IA externa lo expone fuera del control de la empresa."
        ),
    ),
    # ... CBU, credencial/token, correo electrónico
]

def analizar_texto(texto: str) -> ResultadoAnálisis:
    resultado = analizar_con_reglas(texto)
    if resultado:
        return resultado
    resultado = analizar_con_nlp(texto) # punto de extensión spaCy
    if resultado:
        return resultado
    return ResultadoAnálisis(nivel_riesgo="bajo", ...)

```

Fragmento de backend/app/detection.py: reglas por expresiones regulares y punto de entrada único del motor.

El módulo deja declarado explícitamente el punto de extensión `analizar_con_nlp`, que hoy no ejecuta ninguna lógica (devuelve `None`), reservado para la incorporación de la capa de NLP/NER con spaCy descrita en el marco conceptual, sin requerir modificar el resto del backend ni el contrato ya consolidado con la extensión y el dashboard. El resultado de cada análisis se persiste en un registro de eventos en memoria (`store.py`), estructurado para ser reemplazado por una base de datos persistente sin impacto en los endpoints que lo consumen.

Implementación del dashboard de administración con registro de eventos, usuario, tipo de dato detectado e IA destino

El dashboard se implementó en React sobre Vite, consumiendo exclusivamente el endpoint `GET /eventos` del backend, sin acceso directo al registro de eventos, respetando el principio de separación de capas establecido en la Etapa 2. El componente raíz (`App.jsx`) mantiene el estado de los filtros seleccionados por el usuario administrador (usuario, IA de destino, nivel de riesgo) y, ante cualquier cambio, vuelve a consultar al backend con esos parámetros como query string, delegando el filtrado real en el servidor.

```

export async function obtenerEventos(filtros = {}) {
    const params = new URLSearchParams();
    if (filtros.usuario_id) params.set("usuario_id", filtros.usuario_id);
    if (filtros.ia_destino) params.set("ia_destino", filtros.ia_destino);
    if (filtros.nivel_riesgo) params.set("nivel_riesgo", filtros.nivel_riesgo);

    const resp = await fetch(`${BACKEND_BASE_URL}/eventos?${params.toString()}`);
    if (!resp.ok) throw new Error(`Error al consultar eventos: ${resp.status}`);
}

```

```
return resp.json();  
}
```

Fragmento de dashboard/src/api.js: cliente HTTP hacia el backend.

Cumpliendo RF-09, la tabla de eventos (EventsTable.jsx) permite filtrar por usuario, IA de destino y nivel de riesgo, mostrando por cada evento la fecha, el tipo de dato detectado y la acción tomada por el usuario. Cumpliendo RF-10, el panel de métricas (MetricsPanel.jsx) presenta gráficos de barras agregados por nivel de riesgo, por IA de destino y por usuario, implementados con la librería Recharts sobre el mismo conjunto de eventos filtrado.

Acción del usuario

Cuando el sistema detecta riesgo medio/alto/crítico y muestra el modal, el usuario tiene 3 botones para elegir, y esa elección queda guardada como la "acción":

Valor guardado	Qué significa
editar	El empleado apretó "Editar el texto" — volvió atrás a corregirlo
cancelar	Apretó "Cancelar envío" — no mandó el mensaje
continuar	Apretó "Continuar de todos modos" — igual lo mandó, a pesar de la alerta
permitido_automatico	El riesgo fue bajo, ni siquiera le apareció el modal, se mandó directo
— (vacío)	Todavía no se sabe qué hizo (raro, pero puede pasar si algo falla a mitad de camino)

Integración y conexión de la extensión con el backend para validar el envío de datos y la recepción de la respuesta con el análisis de riesgo

Se validó el flujo de extremo a extremo levantando los tres componentes en un entorno local de desarrollo: el backend mediante Uvicorn en el puerto 8000, el dashboard mediante el servidor de desarrollo de Vite, y la extensión cargada en modo desarrollador en Chrome. Para permitir que el dashboard, servido desde un origen distinto al del backend, pudiera consumir la API durante el desarrollo, se habilitó CORS en el backend mediante CORSMiddleware.

La integración se verificó en dos niveles. Primero, a nivel de backend, ejecutando peticiones POST /analizar y GET /eventos directamente contra el servidor local y confirmando que la respuesta respeta el contrato (nivel_riesgo, tipo_dato_detectado, fragmento_detectado, mensaje_contextual, evento_id) y que el evento queda disponible en el listado. Segundo, a nivel de extensión, verificando

en un sitio de IA soportado que un texto con un dato sensible (por ejemplo, un CUIT) dispara el modal de intervención contextual en lugar de enviarse, y que la acción elegida por el usuario queda reflejada en el registro de eventos consultado por el dashboard.

Módulo	Subsistema	Responsabilidad	Tecnología	Depende de
API REST	Backend	Expone POST /analizar, PATCH /eventos/{id} y GET /eventos; habilita CORS para el entorno de desarrollo.	Python, FastAPI	Motor de Detección, Registro de Eventos
Motor de Detección	Backend	Aplica reglas por expresiones regulares sobre el texto recibido; punto de extensión declarado para NLP (spaCy).	Python	Ninguno (módulo autocontenido)
Registro de Eventos	Backend	Persiste cada evento evaluado en memoria y expone el historial consultable por el Dashboard.	Python (in-memory)	Ninguno
Content Script	Extensión	Intercepta el envío en ChatGPT/Gemini/Claude según site-adapters.js y renderiza el modal contextual.	JavaScript (MV3)	Service Worker
Service Worker	Extensión	Orquesta el POST /analizar, persiste usuario_id con chrome.storage y notifica la acción vía PATCH.	JavaScript, fetch	API REST del backend
Cliente API + Vistas	Dashboard	Consume GET /eventos con filtros; renderiza tabla filtrable y panel de métricas.	React, Recharts	API REST del backend

Estado de implementación de los módulos del sistema al cierre de la Etapa 3.

Con los tres componentes verificados de forma conjunta, queda validado el contrato de comunicación consolidado en la Etapa 2 sobre una implementación real, lo que habilita avanzar hacia la Etapa 4: el diseño de los escenarios de prueba con los tres perfiles de empleado definidos en el marco conceptual, y la evaluación empírica de la intervención contextual frente al bloqueo genérico.

Referencias bibliográficas

1. Google Developers. (2024). *Chrome Extensions — Manifest V3 Overview*. Recuperado de <https://developer.chrome.com/docs/extensions/mv3>
2. Chrome Developers. (2026). *What's new in extensions*. Recuperado de <https://developer.chrome.com/docs/extensions/whats-new>
3. Meta. (2024). *React — A JavaScript library for building user interfaces*. Recuperado de <https://react.dev>

4. GreatFrontEnd. (2026). *React vs Angular vs Vue in 2026: Which Framework Should You Learn*. Recuperado de <https://www.greatfrontend.com/blog/react-vs-angular-vs-vue>
5. Google Developers. (2024). chrome.storage API. Recuperado de <https://developer.chrome.com/docs/extensions/reference/api/storage>
6. MDN Web Docs. (2024). Using the Fetch API. Recuperado de https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
7. Ramírez, S. (2024). CORS (Cross-Origin Resource Sharing) — FastAPI. Recuperado de <https://fastapi.tiangolo.com/tutorial/cors/>
8. Recharts. (2024). Recharts — A composable charting library built on React components. Recuperado de <https://recharts.org>
9. Vite. (2024). Vite — Next Generation Frontend Tooling. Recuperado de <https://vite.dev>
10. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, University of California, Irvine].